

AGT Platform

Chantier 1 — Finalisation & Stabilisation

Conception B5 : Base de connaissance & Features

Phase 0 — Audit et Conception des Modèles

Features 1 à 12 sur 24

Document de référence — Session 32

Gabriel (Lead) — 18 mai 2026

AG Technologies — Confidentiel

Suite : features 13 à 24 dans le prochain document

Contents

1	Préambule	2
2	Principes architecturaux validés	2
2.1	Architecture en 3 couches	2
2.2	Les 4 booléens sur le modèle Feature	2
2.3	Système de catalogues — Source canonique unique	3
2.4	Règle frontend transversale	3
2.5	Images des items — Plan V1 / V2	3
3	Modifications du modèle de données	4
3.1	Modèle Feature — 4 nouveaux booléens	4
3.2	Modèle Ressource — 11 nouveaux champs	4
3.3	Modèle Reservation — 6 nouveaux champs + statut	4
3.4	Modèle ChambreType — 2 nouveaux champs	4
3.5	Modèle Commande — 2 nouveaux champs	5
3.6	Modèle ItemCatalogue — 10 nouveaux champs	5
3.7	Modèle QuestionFrequente — 1 nouveau champ	5
3.8	Nouveaux modèles à créer	5
3.8.1	ConsultationFAQ	5
3.8.2	TransfertHumain	5
3.9	Actions bot à créer	6
4	Règles métier transversales	6
4.1	Notifications après réservation ou commande	6
4.2	Politique d’annulation par feature	7
4.3	Livraison — Hors scope Chantier 1	7
4.4	commande_paiement — Hors scope Chantier 1	7
4.5	FAQ — Injection statique à remplacer	7
4.6	Sécurité suivi de commande	7
4.7	Gestion du stock	7
4.8	Confirmation automatique conditionnelle	7
5	Fiches features 1 à 12	8
6	Tableau récapitulatif des actions prioritaires	11
7	Ordre d’exécution recommandé	12

1 Préambule

Ce document est le résultat de l’audit approfondi des 12 premières features du bloc **B5 (Chaîne métier complète)** du Chantier 1. Il sert de **boussole** pour toutes les sessions qui travailleront sur B5.

Comment lire ce document. Chaque fiche feature décrit en français simple ce qui existe déjà, ce qui manque, et exactement ce qu’il faut faire. Rien n’est ambigu. Aucune décision n’est laissée à l’interprétation.

Règle absolue : aucune modification de modèle de données n’est faite sans avoir relu la section correspondante dans ce document. Aucune initiative silencieuse. Toute évolution non prévue ici est soumise à Gabriel avant action.

2 Principes architecturaux validés

2.1 Architecture en 3 couches

Toute activité du bot laisse une trace dans l’une de ces 3 couches. Elles ne se mélangent pas.

Couche	Modèle	Qui écrit	Ce qu’elle capture
Communication	<code>AIConversation</code> / <code>AIMessage</code>	Bot + Client	Le dialogue brut complet
Exécution	<code>AIActionLog</code>	Bot uniquement	Chaque action : type, payload, résultat, durée
Résultat métier	<code>Reservation</code> , <code>Commande...</code>	Bot + Entreprise	L’outcome business à valeur opérationnelle

Règle fondamentale : toute feature active produit **au minimum une vue stats** pour l’entreprise. Pour les features sans Résultat métier (FAQ, catalogues d’orientation), la source des stats est `AIActionLog`. Aucune feature n’est laissée sans visibilité.

2.2 Les 4 booléens sur le modèle Feature

Ces 4 booléens pilotent le comportement du bot et de l’interface pour chaque feature. Ils doivent être positionnés correctement dans le seeder.

Booléen	Signification
<code>entreprise_configure_kb</code>	L'entreprise doit/peut configurer quelque chose dans <code>/knowledge</code> pour que cette feature fonctionne
<code>bot_lit_kb</code>	Le bot lit la KB de cette feature pendant la conversation avant d'agir
<code>bot_ecrit_result</code>	Le bot crée un enregistrement dans une table Résultat métier
<code>entreprise_peut_ecrire_result</code>	L'entreprise peut intervenir sur les résultats (valider, annoter) avec mention de source

Sur le modèle `ResultRecord` (base abstraite de tous les résultats) :

- `source` : bot / entreprise / systeme
- `source_user` : FK User nullable (si source = entreprise)

2.3 Système de catalogues — Source canonique unique

Trois systèmes coexistaient. **Décision définitive** : le Système 3 est la source de vérité unique. Les Systèmes 1 et 2 sont dépréciés et leurs données migrées.

Système	Statut	Action
S1 : <code>MenuCategorie/MenuPlat</code>	Déprécié	Migrer vers <code>ItemCatalogue</code> + conserver le frontend
S2 : <code>CatalogueProduit, CatalogueService, etc.</code>	Déprécié	Migrer + récupérer les champs spécialisés
S3 : <code>Catalogue/ItemCatalogue</code>	Canonique	Enrichir avec les champs manquants

Règle de migration : on ne supprime rien tant que la migration n'est pas validée par Gabriel sur un compte de test. Les anciens endpoints restent actifs pendant la transition.

2.4 Règle frontend transversale

On n'expose sur le frontend **que ce qui fonctionne réellement côté backend**. Aucun bouton qui mène nulle part. Les fonctionnalités non opérationnelles sont masquées ou affichent *"Bientôt disponible"*.

2.5 Images des items — Plan V1 / V2

Version	Approche
V1 — Chantier 1	Bibliothèque d'images statiques préconfigurées par type d'item. L'entreprise choisit dans une liste. Stockées dans <code>public/</code> .
V2 — Chantier 2	Upload d'images réelles par l'entreprise — stockage cloud (S3/Cloudflare), miniatures automatiques.

3 Modifications du modèle de données

3.1 Modèle Feature — 4 nouveaux booléens

```
entreprise_configure_kb      : BooleanField  default=False
bot_lit_kb                  : BooleanField  default=False
bot_ecrit_result            : BooleanField  default=True
entreprise_peut_ecrire_result : BooleanField  default=False
```

Seeder à mettre à jour pour positionner ces booléens sur les 24 features.

3.2 Modèle Ressource — 11 nouveaux champs

```
duree_minutes              : PositiveIntegerField  default=30
delai_min_heures           : PositiveIntegerField  default=0
buffer_apres_minutes       : PositiveIntegerField  default=15
zone                       : CharField(50) choices=[terrasse, interieur,
    salon_vip, autre]
est_accessible_pmr          : BooleanField  default=False
tags                       : JSONField list  default=[]
chambre_type               : ForeignKey(ChambreType, null=True, blank=True)
trajet                     : ForeignKey(CatalogueTrajet, null=True, blank=True)
confirmation_automatique   : BooleanField  default=False
politique_annulation       : CharField choices=[libre, avec_frais,
    non_remboursable] default=libre
config_annulation          : JSONField  default={}
```

`buffer_slot_min` sur `ProfilEntreprise` est **déprécié**. Conservé en base pour compatibilité descendante, non utilisé dans les nouveaux calculs.

3.3 Modèle Reservation — 6 nouveaux champs + statut

```
notif_client_envoyee       : BooleanField  default=False
notif_entreprise_envoyee   : BooleanField  default=False
montant_rembourse          : DecimalField(10,2) null=True, blank=True
date_remboursement         : DateTimeField null=True, blank=True
motif_annulation           : TextField  blank=True
annule_par                 : CharField choices=[client, entreprise,
    systeme] blank=True
```

Ajouter le statut `en_cours` (check-in effectué) pour les features avec durée (chambre, billet).

3.4 Modèle ChambreType — 2 nouveaux champs

```
prix_nuit      : DecimalField(10,2) null=True, blank=True
equipements    : JSONField list  default=[]
```

3.5 Modèle Commande — 2 nouveaux champs

```
numero_commande : CharField(30) unique=True NOT NULL
                  Format : CMD-YYYYMMDD-XXXX (genere a la creation)
notes_client    : TextField blank=True
                  Visible par le bot au client. Distinct de notes (
                    interne uniquement).
```

3.6 Modèle ItemCatalogue — 10 nouveaux champs

```
image_url        : URLField blank=True
temps_preparation_min : PositiveIntegerField null=True, blank=True
allergenes       : JSONField list default=[]
est_disponible_aujourd'hui : BooleanField default=True
stock            : PositiveIntegerField null=True, blank=True
                  null = pas de gestion de stock (gere
                    manuellement)
reference        : CharField(100) blank=True
duree_min       : PositiveIntegerField null=True, blank=True
type_service    : CharField(50) blank=True
action_suivante : CharField choices=[proposer_rdv,
    proposer_inscription,
    proposer_devis, contact_direct,
    proposer_billet, aucune]
                  default=aucune
classes         : JSONField default=[]
                  Format : [{"nom": "Standard", "prix": 5000},
                    {"nom": "VIP", "prix": 8000}]
ville_depart    : CharField(100) blank=True
ville_arrivee   : CharField(100) blank=True
```

3.7 Modèle QuestionFrequente — 1 nouveau champ

```
ordre : PositiveIntegerField default=0
```

3.8 Nouveaux modèles à créer

3.8.1 ConsultationFAQ

```
class ConsultationFAQ(models.Model):
    conversation = FK(AIConversation)
    question_posee = TextField()
    question_matchee = FK(QuestionFrequente, null=True) # null si
    aucune trouvee
    score_similarite = FloatField(default=0)
    reponse_donnee = TextField()
    source = "bot" # toujours bot
    created_at = DateTimeField(auto_now_add=True)
```

3.8.2 TransfertHumain

```
class TransfertHumain(models.Model):
    STATUT = [en_attente, pris_en_charge, resolu]
    conversation = FK(AIConversation)
    agence = FK(Agence)
    motif = TextField()
    statut = CharField(choices=STATUT, default=en_attente)
    assigne_a = FK(User, null=True)
    pris_en_charge_le = DateTimeField(null=True)
    resolu_le = DateTimeField(null=True)
    source = "bot"
    source_user = FK(User, null=True)
    created_at = DateTimeField(auto_now_add=True)
```

3.9 Actions bot à créer

Action	Paramètres	Description
search_faq	query: str	Recherche dans <code>QuestionFrequente</code> par mots-clés. Remplace l'injection statique. Retourne top 3–5 résultats.
get_menu	categorie?	Items actifs de <code>feature=menu_digital</code> .
get_catalogue	query?, categorie?	Items actifs filtrés.
get_services	query?	Items services avec <code>type_service</code> et <code>action_suivante</code> .
get_trajets	depart?, arrivee?	Filtre sur <code>ville_depart/ville_arrivee</code> .
get_room_types	aucun	<code>ChambreType</code> actifs avec prix et équipements.
get_order_status	numero_commande	Vérifie <code>phone WhatsApp == contact.phone</code> avant de retourner le statut.

4 Règles métier transversales

4.1 Notifications après réservation ou commande

S'applique à : `prise_rdv`, `reservation_table`, `reservation_chambre`, `reservation_billet`, `orientation_patient`, `menu_digital`, `catalogue_produits`.

Déclencheur 1 — Création par le bot : notifier l'entreprise + envoyer un accusé de réception au client.

Déclencheur 2 — Changement de statut par l'entreprise : notifier le client (confirmé / annulé).

Implémentation : signaux Django `post_save`. Réutilise l'action `send_email` existante. Champs `notif_client_envoyee` et `notif_entreprise_envoyee` sur `Reservation` pour éviter les doublons.

4.2 Politique d'annulation par feature

Feature	Politique par défaut	Configurable
<code>prise_rdv</code>	libre	Oui
<code>reservation_table</code>	libre	Oui
<code>reservation_chambre</code>	avec_frais	Oui
<code>reservation_billet</code>	non_remboursable	Oui
<code>orientation_patient</code>	libre	Oui

Remboursement = Transaction de type `credit` référençant la Reservation annulée.

4.3 Livraison — Hors scope Chantier 1

Statut `livree` existe en base mais masqué côté frontend. Statuts visibles pour `menu_digital` et `catalogue_produits` : `en_attente` / `confirmee` / `en_preparation` / `prete` / `annulee`.

4.4 commande_paiement — Hors scope Chantier 1

Requiert un provider externe opérationnel. Badge "*Bientôt disponible*" en Chantier 1. Implémentation en Chantier 2.

4.5 FAQ — Injection statique à remplacer

Priorité haute. Le `ContextBuilder` injecte actuellement toute la FAQ dans le `system prompt` (Bloc 2). Cela ruinera les tokens si une entreprise a 1 000 questions. **Action :** supprimer la FAQ du Bloc 2, créer l'action `search_faq(query)` dynamique.

4.6 Sécurité suivi de commande

Avant de partager tout statut de commande, le bot vérifie : `Commande.contact.phone == conversation.contact.phone`. En cas d'échec : TransfertHumain automatique. `numero_commande` obligatoire, non nullable, généré à la création.

4.7 Gestion du stock

Le bot **lit** le stock pour vérifier la disponibilité. Il ne le **modifie jamais**. Gestion exclusivement manuelle par l'entreprise.

4.8 Confirmation automatique conditionnelle

`Ressource.confirmation_automatique = True` ne confirme automatiquement **que si le paiement est effectué**. Sans paiement : statut reste `en_attente`.

5 Fiches features 1 à 12

Feature 1 — chatbot_whatsapp

Description métier : Canal d'entrée WhatsApp. Le client envoie ses messages, le bot répond. C'est la plomberie de tout le système.

Ce qui existe : `WahaSession` (connexion QR, statut, numéro), `AIConversation`, `AIMessage`, `send_text` via WAHA.

Booléens Feature :

```
entreprise_configure_kb=False | bot_lit_kb=False |
bot_ecrit_result=True (AIActionLog) | entreprise_peut_ecrire_result=False
```

Actions à mener :

1. Vérifier que `check_quota("chatbot_whatsapp")` est le **premier appel** du pipeline de traitement de tout message entrant.
2. Créer le modèle `TransfertHumain` (voir §2.8).
3. Vérifier que chaque transfert humain crée un `TransfertHumain` avec agence, motif, statut `en_attente`.

Feature 2 — faq

Description métier : L'entreprise rédige ses questions/réponses fréquentes. Le bot les consulte à la demande pour répondre aux clients.

Ce qui existe : `FAQ` (conteneur), `QuestionFrequente` (questions avec catégorie), tab `FAQ` dans `/knowledge` — une seule FAQ affichée (sous-exploité).

Booléens Feature :

```
entreprise_configure_kb=True | bot_lit_kb=True
| bot_ecrit_result=True (ConsultationFAQ) |
entreprise_peut_ecrire_result=False
```

Actions à mener :

1. Ajouter champ `ordre` sur `QuestionFrequente` (migration).
2. Corriger l'interface `/knowledge` pour afficher plusieurs FAQ thématiques distinctes.
3. **Priorité haute :** supprimer l'injection statique de la FAQ dans le `ContextBuilder` (Bloc 2). Créer action `search_faq(query)`.
4. Créer modèle `ConsultationFAQ` (voir §2.8). Logger chaque appel.

Feature 3 — prise_rdv

Description métier : Un client prend rendez-vous via WhatsApp. Le bot vérifie les disponibilités et crée le RDV automatiquement.

Ce qui existe : `Ressource` (`type=praticien/salle`), `DisponibiliteRessource`, `Reservation`, actions `check_disponibilite` et `create_reservation`. Règle R14 active.

Booléens Feature :

```
entreprise_configure_kb=True      |      bot_lit_kb=True      |
bot_ecrit_result=True | entreprise_peut_ecrire_result=True
```

Actions à mener :

1. Ajouter sur `Ressource` : `duree_minutes` (défaut 30), `delai_min_heures` (défaut 0), `buffer_apres_minutes` (défaut 15). Ces 3 champs profitent à toutes les features de réservation.
2. Créer tab "**Disponibilités**" dans `/knowledge`, visible si `prise_rdv` active.
3. Mettre à jour `check_disponibilite` pour intégrer `buffer_apres_minutes`.
4. Page Résultats : filtrer par `feature.slug = "prise_rdv"`.
5. Boutons de changement de statut sur la page Résultats.

Feature 4 — reservation_table

Description métier : Un client réserve une table au restaurant via WhatsApp. Le bot vérifie et crée la réservation.

Ce qui existe : Même infrastructure que `prise_rdv` (`Ressource` type=table). Aucun tab frontend dédié.

Booléens Feature :

```
entreprise_configure_kb=True      |      bot_lit_kb=True      |
bot_ecrit_result=True | entreprise_peut_ecrire_result=True
```

Actions à mener :

1. Créer tab "**Tables**" dans `/knowledge`, visible si `reservation_table` active.
2. Ajouter sur `Ressource` : `zone`, `est_accessible_pmr`, `tags`.
3. Mêmes actions que `prise_rdv` pour buffer, Résultats, boutons.

Feature 5 — reservation_chambre

Description métier : Un client réserve une chambre d'hôtel via WhatsApp. Le bot présente les types disponibles et crée la réservation.

Ce qui existe : `ChambreType` (catégories), `Ressource` (type=chambre), tab "Chambres" dans `/knowledge`. Pas d'interface pour les chambres individuelles.

Actions à mener :

1. Ajouter sur `ChambreType` : `prix_nuit`, `equipements`.
2. Ajouter sur `Ressource` : `chambre_type` FK.
3. Créer action bot `get_room_types`.
4. Créer interface pour les chambres individuelles dans `/knowledge`.
5. Ajouter statut `en_cours` sur `Reservation`.
6. Politique annulation : `avec_frais` par défaut.

Feature 6 — reservation_billet

Description métier : Un client réserve une place dans un transport via WhatsApp. Le bot présente les trajets et émet le billet.

Ce qui existe : `CatalogueTrajet` (lignes), `Ressource` (type=trajet), `Reservation`.

Actions à mener :

1. Ajouter sur `Ressource : trajet FK`, `confirmation_automatique` (True pour transport — confirme si paiement reçu).
2. Ajouter sur `Reservation : numero_billet`, `siege`.
3. Politique annulation : `non_remboursable` par défaut.
4. Ajouter statut `en_cours` (voyage en cours).
5. Créer interface pour les véhicules dans `/knowledge`.

Feature 7 — menu_digital

Description métier : Le restaurant publie son menu. Un client consulte et commande directement via WhatsApp.

Ce qui existe : S1 (déprécié : `MenuCategorie/MenuPlat`), S3 (canonique : `ItemCatalogue`), `Commande` + `LigneCommande`. Tab "Menu" branché sur S1.

Actions à mener :

1. **Migration S1 → S3 :** rebrancher tab "Menu" sur `ItemCatalogue`. Migrer les données `MenuPlat` → `ItemCatalogue`. Valider avec Gabriel sur un compte de test.
2. Ajouter sur `ItemCatalogue` : `image_url`, `temps_preparation_min`, `allergenes`, `est_disponible_aujourd_hui`.
3. Images V1 : bibliothèque statique (§1.5).
4. Créer action bot `get_menu(categorie?)`.
5. Masquer statut `livree` côté frontend (§3.3).

Feature 8 — commande_paiement — HORS SCOPE C1

Description métier : Passer et payer une commande directement via WhatsApp avec un provider de paiement intégré.

Décision : Différée au Chantier 2. Requier un provider externe opérationnel (Orange Money, MTN MoMo).

En Chantier 1 : feature visible dans la marketplace, badge "*Bientôt disponible*", non activable.

Feature 9 — catalogue_produits

Description métier : Une boutique publie son catalogue. Un client consulte et commande via WhatsApp. Pas de paiement immédiat en C1.

Ce qui existe : S2 (déprécié : `CatalogueProduit` avec `stock`, `reference`), S3 canonique sans ces champs.

Actions à mener :

1. **Migration S2 → S3 :** récupérer `stock` et `reference`.
2. Ajouter sur `ItemCatalogue` : `stock` (nullable), `reference`.
3. Le bot lit `stock` uniquement. Ne le modifie jamais (§3.7).
4. Créer action bot `get_catalogue(query?, categorie?)`.

Feature 10 — suivi_commande

Description métier : Un client demande l'état de sa commande. Le bot la retrouve et donne le statut en temps réel, sans intervention humaine.

Ce qui existe : `Commande` avec statuts, `LigneCommande`, `Contact` (phone connu depuis WhatsApp).

Actions à mener :

1. Ajouter `numero_commande` : obligatoire, unique, format `CMD-YYYYMMDD-XXXX`, généré automatiquement.
2. Ajouter `notes_client` : texte visible par le bot, distinct de `notes` (interne).
3. Créer action `get_order_status(numero_commande)` avec vérification : `Commande.contact.phone == conversation.contact.phone`. Échec → `TransfertHumain` automatique.

Feature 11 — catalogue_services

Description métier : Une entreprise de services liste ses prestations. Le bot les présente et oriente vers la prochaine action selon la configuration.

Ce qui existe : S2 (déprécié : `CatalogueService` avec `duree_min`). Usage multi-secteurs : PME, Santé, Custom.

Actions à mener :

1. **Migration S2 → S3 :** récupérer `duree_min`.
2. Ajouter sur `ItemCatalogue` : `duree_min`, `type_service` (libre), `action_suivante` (choices).
3. Créer action `get_services(query?)`. Le bot enchaîne selon `action_suivante` (ex : si `proposer_rdv`, appelle `check_disponibilite`).
4. Interface pour configurer `type_service` et `action_suivante` par prestation.

Feature 12 — catalogue_trajets

Description métier : Une compagnie publie ses lignes, horaires et tarifs. Le bot présente les trajets et oriente vers la réservation de billet.

Ce qui existe : S2 (déprécié : `CatalogueTrajet` avec `horaires_depart` `JSONField`). Feature complémentaire à `reservation_billet`.

Relation avec `reservation_billet` : `CatalogueTrajet` = la ligne (horaires, tarifs). Ressource `type=trajet` = le véhicule physique. Le FK `Ressource.trajet` → `CatalogueTrajet` assure le lien.

Actions à mener :

1. **Migration S2 → S3 :** récupérer les champs `CatalogueTrajet`.
2. Ajouter sur `ItemCatalogue` : `classes` (`JSONField`), `ville_depart`, `ville_arrivee`.
3. `action_suivante` = "proposer_billet" — fixe pour cette feature.
4. Créer action `get_trajets(depart?, arrivee?)`.

6 Tableau récapitulatif des actions prioritaires

Priorité	Action	Feature(s)	Type
Haute	Corriger <code>ContextBuilder</code> : supprimer FAQ Bloc 2, créer <code>search_faq</code>	FAQ	Backend
Haute	Ajouter <code>numero_commande</code> (non nulle, unique)	suivi_commande	Migration
Haute	Ajouter les 4 booléens sur <code>Feature</code> + seeder	Toutes	Migration
Haute	Ajouter les 11 champs sur <code>Ressource</code>	rdv, table, chambre, billet	Migration
Moyenne	Migration S1/S2 → S3 (<code>ItemCatalogue</code>)	menu, produits, services, trajets	Migration
Moyenne	Créer tab "Disponibilités" dans <code>/knowledge</code>	prise_rdv	Frontend
Moyenne	Créer tab "Tables" dans <code>/knowledge</code>	reservation_table	Frontend
Moyenne	Corriger tab FAQ multi-FAQ	faq	Frontend
Moyenne	Créer modèle <code>TransfertHumain</code>	chatbot_whatsapp	Backend
Moyenne	Créer modèle <code>ConsultationFAQ</code>	faq	Backend
Basse	Ajouter 10 champs sur <code>ItemCatalogue</code>	menu, produits, services, trajets	Migration
Basse	Créer les 7 actions bot	Multiples	Backend
Basse	Bibliothèque images statiques V1	menu, produits	Frontend
Basse	Badge "Bientôt disponible" sur <code>commande_paiement</code>	commande_paiement	Frontend

7 Ordre d'exécution recommandé

Étape 1 — Migrations (faire en un seul lot). Feature (4 booléens + seeder) · Ressource (11 champs) · Reservation (6 champs + statut `en_cours`) · ChambreType (2 champs) · Commande (`numero_commande` + `notes_client`) · ItemCatalogue (10 champs) · QuestionFrequente (ordre) · Créer `TransfertHumain` · Créer `ConsultationFAQ`.

Étape 2 — Backend actions. `search_faq` (débloquer tests FAQ) · `get_menu` · `get_catalogue` · `get_services` · `get_trajets` · `get_room_types` · `get_order_status` (avec vérification sécurité).

Étape 3 — Migration données S1/S2 → S3. Valider sur compte de test avec Gabriel · Supprimer S1/S2 après validation.

Étape 4 — Frontend. Rebrancher tab Menu/Catalogue sur ItemCatalogue · Créer tabs Disponibilités + Tables · Corriger tab FAQ multi-FAQ · Badge Bientôt disponible sur `commande_paiement`.

Document généré en session 32 — Gabriel (Lead) — 18 mai 2026

AG Technologies — Confidentiel

Suite : features 13 à 24 dans le prochain document de session